The Combinator Soup: Monoculture Collapse onto Self-Reproducing Absorbers, and the Conserving Reactor That Extinguishes Them

Working paper v0.3, Project secret3, August 11, 2026

Abstract

Two reactors over the two-combinator calculus, and one question between them: can organization emerge from separation and sharing alone? The first reactor is an unconstrained flow vessel, 600 closed terms over S and K, collisions by application, fuel-bounded reduction, random dilution. Three runs from three initial conditions all end in total monoculture: the winners are self-reproducing absorbers that duplicate any partner, erase every copy, and rebuild themselves, and both discovered species are upgraded to compiler-checked theorems, their four-step and eight-step cycles and normal-form status verified. The selection mechanism is exact: identity-like terms are altruists whose every collision births the partner's species; absorbers are perfectly selfish; unconstrained selection finds the selfish. The second reactor takes the remedy the collapse names, make duplication cost and erasure pay: an atom-conserving chemistry in which an S firing must recruit a free copy of its shared argument by random encounter and releases an S atom, a K firing releases the discarded argument and a K atom, and total mass, with S and K counted separately, is invariant by compiled theorems. Built as a closed vessel this chemistry freezes, twice, into a glass of unique molecules, and the two frozen regimes are diagnosed; driven by reversible condensation it reaches a sustained steady state through 180,000 ticks. In the driven vessel the absorbers of the first reactor never establish at all, the duplication tax is measured at roughly one success per nine attempts, and the species that persist as a standing crop are exactly the identity family the first reactor exterminated, including a standing population of S K K, identity itself. A controlled null, the same vessel and seeds with the reactive channels disabled, isolates the chemistry's fingerprint: the reactions enrich the free atom pool three to four fold and roughly triple the two-atom composite sector, the logged reaction network is entirely food-reachable with roughly half its species on recycling cycles through the atoms, and per-species claims at three atoms are shown to lie below this vessel's statistical resolution.

1. Introduction

Algorithmic chemistry asks what happens when programs are also molecules: a population of terms, collisions that apply one to another, products returned to the vessel. Fontana and Buss built the original reactor over lambda terms and found that, unfiltered, it collapses onto trivial self-maintaining functions, which they had to remove by hand before organizations could appear [5]. Kruszewski and Mikolov ran the idea directly over S and K with a mass-conserving reaction scheme designed precisely to tax duplication, and observed emergent autocatalytic patterns [6]. This paper runs the unconstrained flow reactor over S and K, on the engine discipline of the companion papers [1, 2], and reports what the collapse looks like when nothing prevents it: which species win, how fast, by what mechanism, and what the winners are as mathematical objects.

This line of experiments asks one question: can organization emerge from separation and sharing alone? The present paper plays three roles in that question, and states them up front. First, the null model: any future claim that organization emerged is meaningful only against the baseline of what

the unconstrained chemistry does, and Sections 2 to 5 establish that baseline exactly. It does not explore; it fixates, logistically and totally, onto a selfish absorber, from every tested start. Second, the verified anatomy: the winning species are identified exactly and their dynamics promoted to compiler-checked theorems, which is, to our knowledge, the first theorem-grade treatment of species discovered by an artificial-life reactor, and the wedge this program brings to the field. Third, the calibration: the collapse names its own remedy, make duplication cost and erasure pay, and Sections 6 to 8 build that reactor and report its answer, with Sections 9 and 10 adding the controlled null and the analysis of the logged reaction network. Under conservation the absorber strategy never establishes at all, the vessel must be driven or it freezes, and the standing crop of the driven vessel is the identity family that the unconstrained reactor exterminated.

The contributions, in order: a fully deterministic flow reactor whose every number is reproducible from a stated seed (Section 2); the takeover data of three runs, including a fine-grained logistic fixation curve and one two-absorber competition (Section 3); the exact anatomy of both winning species as compiled theorems, tauAbsorbs, upsAbsorbs, and the normal-form facts (Section 4); the selection mechanism in one sentence, connected to the occurrence ledger of [3] (Section 5); an atom-conserving chemistry whose reaction balances are themselves compiled theorems, with two frozen regimes diagnosed on the way to a driven steady state (Sections 6 and 7); and the conserving vessel's verdict on the absorbers, with the duplication tax measured (Section 8).

2. The reactor

The population is an array of 600 closed terms over S and K. One collision: draw an ordered pair (i, j) uniformly, form the application of term i to term j, and reduce in normal order with a fuel of 250 steps and an intermediate size guard of 300 leaves. If the pair's combined size exceeds 80 leaves, if fuel or guard is exhausted, or if the product exceeds 60 leaves, the collision is elastic and nothing changes. Otherwise the product replaces a uniformly random population slot: the dilution flow of a well-mixed reactor at constant volume. Randomness is a 64-bit linear congruential generator with the PCG multiplier, seeded per run; every table below is reproducible from the seed. Run A starts from 600 random trees of two to seven leaves (seed 42), run B from the 22-species lattice of all terms with at most three leaves, cycled to 600 (seed 7), run C from a second random population (seed 1234). The reactor source, engine-class rather than proof-class, is Appendix B; the theorems of Section 4 live in the shared proof module, Appendix C.

3. Results

Initial conditions. Run A opens with 362 distinct species, mean size four; the most common are the four two-leaf terms (SK at 33, KK at 30, SS at 26, KS at 21). Run B opens with its designed 22 species at near-uniform 27 to 28 copies each. Run C opens with 356 distinct species. In all three, the behavioral census at time zero counts roughly fifty identity-like and 160 to 180 constant-like members under the two-probe test of Appendix A.

The takeover, fine grain (run A). With checkpoints every 2,000 collisions:

collisions	distinct	tau count	elastic in window	note
0	362	0		random start
2,000	366	below top eight	91	atoms S, K lead; identity-like peaks at 67
4,000	378	56	318	tau leads; a rival lineage holds 6
6,000	176	338	219	satellites K(tau), S(tau) appear
8,000	23	560	25	diversity collapsing

10,000	5	596	0	last four strays, all containing tau
12,000	1	600	0	fixation

The curve is logistic: 9 percent, 56, 93, 99, 100 across four windows. The transient satellite ecology consists almost entirely of terms containing tau as a subterm (K(tau), S(tau), S(S(tau))(tau), and larger), produced by collisions in which tau was the right partner; the dilution flow washes them out. From 12,000 to 120,000 collisions nothing changes, every window is elastic-free, and the production spectrum at the end is total: 20,000 of 20,000 sampled pairs from the final population produce tau.

Run B. The lattice start fixates on the same species: by the first coarse checkpoint at 30,000 collisions the population is 600 copies of tau, and it stays there through 120,000, spectrum again 20,000 of 20,000.

Run C, a two-absorber race. The second random start fixates on a different winner: upsilon = S(S(S(KK)(SSK))(K(S(S(KK)(SSK)))), sixteen leaves. At 30,000 collisions the population holds exactly two species, upsilon at 564 and tau itself at 36; by 60,000 upsilon owns all 600 slots and holds them, spectrum 20,000 of 20,000. The founder decided the race: both species are absorbers in the sense of Section 4, every collision they lead is a birth for their own kind, so the dynamics between them is drift weighted by abundance, and upsilon was established first.

4. The winners, as theorems

Both winning species are self-reproducing absorbers, and both claims are now compiler-checked in the shared proof module, over the same inductive Steps relation that carries the companion papers' results [1, 2].

tau. Write ssk for SSK. Then tau = S(K ssk)(K(K ssk)), and for every term x whatsoever the cycle closes in four steps: the S firing hands x to both branches, the two K firings erase both copies, and the final S firing rebuilds tau from its own debris.

```
tauAbsorbs : (x : Term) -> Steps (App tau x) tau
tauAbsorbs x =
  Then RedS (Then (CtxL RedK) (Then (CtxR RedK) (Then RedS Done)))
```

The derivation is the proof: one constructor per step, the compiler checking every endpoint. Two corollaries ride along: tauFix, the monoculture's only reaction tau tau to tau, and tauNormal, the fact that tau itself takes no weak step, so the fixated population is a population of normal forms and every collision terminates in the four steps above. The absorption census per collision: two S firings, two K firings, the partner duplicated once and erased twice over.

upsilon. The rival of run C absorbs in eight steps, four S firings and four K firings; the partner enters three copies and every one is erased by a K firing before it is ever reduced, so the cycle closes for arbitrary x, terminating or not:

```
upsAbsorbs : (x : Term) -> Steps (App ups x) ups
```

with upsFix and upsNormal alongside. The two species are behaviorally identical at the reactor's resolution, absorb anything, rebuild self, and differ only in metabolic cost, four steps against eight. The reactor does not price steps, so the cheaper metabolism confers no advantage; Section 5 returns to this.

5. The mechanism, and the ledger

Why absorbers, and why not the identities that led the early census? The mechanism fits in one sentence: in this reactor, a term's reproductive success is the rate at which collisions produce copies of it, and an identity-like term gives every one of its collisions away, since id x reduces to x and the birth goes to the partner's species, while an absorber keeps every one, since abs x reduces to abs. Identity is an altruist; the absorber is perfectly selfish; unconstrained selection finds the selfish. The early identity peak in run A (67 members at 2,000 collisions) is the altruists briefly riding the diversity they themselves feed, before the first absorber lineage outgrows everything.

The occurrence ledger of the linearity paper [3] prices each absorption exactly. Every cycle is the same gesture the calculus performs in the small: duplicate the other (S), erase every copy (K), restore the self (S). For tau the per-collision account is two duplicating firings and two erasing ones; for upsilon, four and four. Nothing about the partner survives; the compiled chains make the census exact, firing by firing.

This replicates, inside S and K, the collapse that made Fontana and Buss filter their reactor by hand [5], and it locates the remedy the literature already found: make duplication cost something. In the mass-conserving scheme of Kruszewski and Mikolov an S firing must recruit a physical copy of its argument from the vessel and a K firing releases material back [6], which is exactly a tax on the absorber's metabolism and a wage for the altruist's. The sequel is no longer queued: Sections 6 to 8 build the conserving reactor and report its answer.

6. The conserving reactor

The second chemistry makes the two moves pay. Its molecules are the same closed terms over S and K, its vessel a multiset, and its reactions balance like chemical equations, with the balances themselves compiled theorems in the shared proof module (Appendix E). A condensation joins two molecules into their application, conserving atoms on the nose (condMass). A K reaction fires a K redex anywhere in a molecule and releases both the discarded argument and one free K atom into the vessel; the redex weighs exactly its product plus the freed argument plus the released atom (kBalanced), and the balance holds for S atoms and K atoms counted separately (kBalS, kBalK). An S reaction is taxed: to fire, it must recruit one free copy of its shared argument from the vessel, found by random encounter, up to 24 uniform probes, so that recruitment probability tracks concentration; the redex plus the recruited copy weighs exactly its product plus one released S atom (sBalanced, sBalS, sBalK). Erasure pays; duplication costs. Total mass and each element separately are invariant, and every run below verifies mass 6,000 at every checkpoint, the runtime mirror of the compiled balances.

The vessel opens on genesis conditions: 3,000 free S atoms and 3,000 free K atoms, nothing else. Each tick draws an action, condensation with probability two in six, cleavage one in six, a reaction attempt three in six; a reaction attempt draws a molecule, enumerates its reactive sites, picks one uniformly, and fires it if it is a K site or a successfully recruited S site. Products above 24 atoms block condensation. Everything is deterministic from the seed.

The protocol reached this form through two instructive failures, both reported because each is a finding about closed conserving chemistries. The first variant fired only the leftmost redex of a drawn molecule and had no cleavage. It froze: by 60,000 ticks the vessel of run one held 170 molecules, every one structurally unique, zero free atoms of either kind, and all reaction counters at a standstill, because a single blocked S site at the head of a molecule occludes every K site behind it, and condensation is a one-way ratchet that locks atoms into unique boulders no recruitment can ever match. The second variant freed the site choice, any reactive site may be the one that reacts, and lived visibly longer: at 30,000 ticks a real economy, 132 free K, 59 free S, standing populations

of the two-atom species; by 90,000 ticks, frozen again, 264 unique molecules, zero free atoms. Site freedom cures the occlusion; the ratchet remains. A closed vessel with irreversible condensation has a glass for a ground state, and it finds it. The cure is throughput: make condensation reversible. The delivered protocol adds cleavage, a drawn molecule splits at its root into its two children, trivially mass-conserving, at rate one in six, and the vessel becomes driven rather than closed.

7. Results of the driven vessel

Three runs: seed 11 and seed 77 at condensation cap 24, and seed 11 at cap 40, each 180,000 ticks with checkpoints every 30,000. All three sustain. Run one settles into a steady state around 550 to 620 molecules and 345 to 360 distinct species, with persistent free atom pools (44 to 140 free S, 110 to 233 free K across checkpoints) and continuous reaction flux in every window to the end: cumulatively 8,719 K firings and 3,562 S firings by 180,000 ticks, still climbing between the last checkpoints. Mass reads 6,000 at every checkpoint of every run. The established core at the end of run one, species with at least five copies and at least two atoms, is SK at 15, KK at 14, KS at 6, and SKK at 5; run two's core is KK at 16, SK at 14, KS at 8, SS at 8, with SKK just under threshold at 3. The looser cap of run three drains the economy into larger blocked molecules, a thinner but still living state, 293 molecules, continuing flux, one core species.

The duplication tax is now a measured number. At cap 24, S attempts succeed at 11.8 and 10.9 percent in the two runs (3,562 of 30,155 and 3,320 of 30,467 attempts); at cap 40, 7.4 percent. Erasure, by contrast, is unconditional: every drawn K site fires. The realized firing mix runs about 2.4 K firings per S firing.

And the absorbers: zero. In all three driven runs, across every checkpoint, the count of tau is zero and the count of upsilon is zero. The species that owned the unconstrained reactor totally never establish a single copy in the conserving one.

8. What conservation selects

The extinction has an arithmetic, not just a moral. In the flow reactor, an absorber's collision was a birth: tau met x, the product tau replaced a random molecule, and the parent tau persisted, net plus one. In the conserving vessel the meal consumes the eater: tau must first condense with x into the single molecule tau x, and the reaction sequence, if its S sites can recruit at all, returns one tau from one tau, net zero, minus the recruitment tax on every duplicating step along the way. The selfish strategy is not merely disadvantaged; its amplification arithmetic is structurally disabled by conservation. What persists instead is a standing crop of the smallest useful species, the two-atom molecules and, in run one, S K K itself, identity, maintained at population five in a vessel of 570: the altruists of the first reactor, whose collisions give their products away, are the durable citizens of the balanced one. We state this carefully: persistence of small species at these counts is compatible with an equilibrium ensemble maintained by the condensation and cleavage flux, and the null-model comparison, whether any composite species stands above its random-assembly expectation, is the next instrumentation this line needs. What is not an artifact is the contrast: total monoculture on one side of the conservation law, absorber extinction and a diverse driven steady state on the other.

The three protocol stages also carry a lesson worth stating in thermodynamic language, since the chemistry earned it. A closed vessel with an irreversible ratchet freezes into a glass, twice, by two different routes; a vessel with reversible aggregation and continuous throughput sustains flux, diversity, and standing populations indefinitely. Organization in this chemistry is not a property of the rules alone; it is a property of being driven. This is the register in which the entropy-budget experiment E7 will speak [4], and the conserving reactor now provides it a vessel where every erasure and every duplication is already a countable, balanced event.

9. The null-model census

Section 8 left one question explicitly outstanding: are the standing populations of the driven vessel a fingerprint of the chemistry, or only the equilibrium of aggregation? The instrument is an operational null: the same vessel, the same seeds, the same tick schedule, with the reactive channel disabled, so that only condensation and cleavage act at their unchanged rates. Whatever the null grows is random assembly; whatever the full chemistry grows beyond it is the work of the two moves.

Two seed pairs, 180,000 ticks each, compared at the final checkpoint:

quantity	full, seed 11	null, seed 11	full, seed 77	null, seed 77
molecules N	571	442	558	431
distinct species	345	356	338	353
free K atoms	128	39	111	26
free S atoms	44	31	60	37
two-atom sector (SK + KK + KS + SS)	39	18	46	15
S K K	5	at most 1	3	3
SS	4	5	8	8 in full, 4 in null

Three signals are robust across both seed pairs. The chemistry pumps the atom pool: free K stands 3.3 and 4.3 times above null, free S 1.4 and 1.6 times, and the asymmetry matches the firing mix, about 2.5 K firings per S firing, since every K firing releases a K atom and every S firing an S atom. The chemistry pumps the small-composite sector: the pooled two-atom species stand 2.2 and 3.1 times above null, the ejecta of K firings inside larger molecules. And the chemistry fragments: the full vessel holds about thirty percent more molecules at the same mass. One would-be signal is honestly retired: at three atoms and populations of five, per-species enrichment is below this vessel's resolution. S K K reads five against at most one at seed 11 and three against three at seed 77; SS reads under parity at one seed and double at the other. The standing crop of the identity family is real as a crop; whether any single three-atom species is manufactured above its assembly expectation needs longer runs, more seeds, and time-averaged censuses, and that continuation is now precisely specified.

10. The network, its closure, and the recycling core

Every run of this version logs its reaction network: each condensation, cleavage, and firing contributes producer-to-product edges with multiplicities, about a hundred thousand distinct edges per full run. Two analyses were promised by the agenda under the name of Hordijk and Steel's RAF theory, and the first honesty is definitional: this chemistry has no catalysis, every reaction being spontaneous, so the reflexively autocatalytic condition degenerates and the RAF question reduces to food-generated closure, which we compute, together with the structure the closure cannot see, the cycles.

The closure is total. In all four runs, every species present at the final checkpoint is reachable from the food set, the two atoms, through the observed network: unreachable count zero, a sanity theorem about the vessel's history. The cycles are the finding. Call a species part of the recycling core if the observed network both reaches it from the atoms and returns from it to the atoms. The full runs hold 156 and 170 of roughly 340 species in the core; the null runs hold more, 195 and 192 of roughly 355, because condensation and cleavage are exact inverses and place most of what they build on a cycle by construction, while the reactive channels add one-way roads. The metabolism concentrates at the bottom: the busiest edge in both full runs is the condensation of two K atoms into KK,

near 5,700 firings, with its cleavage back near 2,150, the vessel's idle breathing, and the S-side analogue second. The picture Sections 7 to 9 assembled now has its network form: a driven steady state whose mass circulates through the atoms, whose chemistry taxes duplication, pays for erasure in atoms and fragments, and visibly enriches exactly the pools those payments feed.

11. Limitations

Three seeds and two caps are what they are; the takeover shape of the flow reactor and the steady state of the driven vessel are consistent across them, but the parameter space, cleavage rate, probe count, action mix, is barely opened. The recruitment rule matches by exact structural identity, the strictest possible chemistry; softer matching would tax duplication less. The established-core threshold is a reporting choice. The null comparison of Section 9 rests on two seed pairs and single-checkpoint counts; its atom-pool and sector-level signals are large enough to survive that, its three-atom per-species readings are not, and the paper says so. The reactors are engine-class code, covering rather than total; every claim promoted to a theorem, the absorber cycles, the normal forms, the reaction balances, lives in the total proof module, and the split is deliberate. The caps bound the reachable chemistry in both reactors; the discovered absorbers sit far below every cap, and the frozen states of the closed conserving vessel were reached under two different site policies, which is some evidence neither result is a boundary artifact.

12. Conclusion

One calculus, two vessels, two verdicts. Unconstrained, the chemistry of separation and sharing fixates: a selfish absorber, exactly analyzable and now exactly analyzed, rides a logistic wave to total monoculture and stays forever. Conserving, the same chemistry reverses the verdict: the absorber never establishes a single copy, duplication succeeds once in nine attempts and pays for itself in substrate, erasure pays the vessel back in parts and atoms, and what persists is a driven steady state whose standing crop is the identity family, with S K K itself holding a population. Between the two vessels sits one law, the conservation of atoms, compiled as four balance theorems, and one requirement, throughput, learned from two frozen glasses on the way. The line's question, whether organization can emerge from separation and sharing alone, now has its instruments: a null model, a balanced vessel that lives, and a proof module that turns whatever the vessels discover into theorems. The null census is delivered, and it sharpens the question rather than closing it: the chemistry's fingerprint is real at the level of pools and sectors, and the resolution needed for per-species claims is now a specified instrument, longer runs, more seeds, time-averaged counts. The phase diagram of the chemistry, cleavage as temperature, is next.

Appendix A: Methods

The flow reactor is Soup.idr (Appendix B), the conserving reactor, in its third version with the null mode and the network logger, Soup2.idr (Appendix D), both compiled with Idris2 0.8.0 [7]; each full experiment set completes in seconds and is bit-for-bit reproducible from the stated seeds. The network analysis, closure by search from the food set and the recycling core as the meet of forward and backward reachability, runs inside the same module after each experiment. The behavioral census of the flow reactor classifies t as identity-like if t applied to S reduces to S and t applied to K reduces to K within fuel, constant-like if both probes reduce to one and the same normal form, by rendered comparison. The conserving runs verify mass 6,000 at every checkpoint, the runtime mirror of the compiled balances. The theorems of Sections 4 and 6 are in the shared proof module Proofs.idr of the project, now 1,261 lines, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification. Appendix C reproduces the absorber section verbatim and Appendix E the conservation section; the full module appears in the companion papers [1, 2].

Appendix B: The reactor

```
||| Project secret3, experiment E6 : the combinator soup.
||| A well-mixed flow reactor over {S, K}: fixed population, random
||| binary collisions (application), fuel-bounded normal-order
||| reduction, random dilution. Deterministic via seeded LCG.
||| Engine-class code (covering, IO); verified claims live elsewhere.
module Soup

import Data.IOArray
import Data.SortedMap
import Data.List
import Data.String
import Data.Maybe

%default covering

data Tm = Sc | Kc | Ap Tm Tm

render : Tm -> String
render Sc = "S"
render Kc = "K"
render (Ap a b) = render a ++ wrap b
  where wrap : Tm -> String
        wrap Sc = "S"
        wrap Kc = "K"
        wrap t = "(" ++ render t ++ ")"

tsize : Tm -> Nat
tsize Sc = 1
tsize Kc = 1
tsize (Ap a b) = tsize a + tsize b

step : Tm -> Maybe Tm
step (Ap (Ap Kc a) b) = Just a
step (Ap (Ap (Ap Sc a) b) c) = Just (Ap (Ap a c) (Ap b c))
step (Ap a b) =
  case step a of
    Just a' => Just (Ap a' b)
    Nothing => case step b of
      Just b' => Just (Ap a b')
      Nothing => Nothing
step _ = Nothing

||| normal order with fuel; intermediate size guard every 8th step
wnf : Nat -> Tm -> Maybe Tm
wnf fuel t = go fuel 8 t
  where go : Nat -> Nat -> Tm -> Maybe Tm
        go Z _ u = Nothing
        go (S f) Z u = case step u of
          Nothing => Just u
          Just u' => if tsize u' > 300 then Nothing else go f 8 u'
        go (S f) (S g) u = case step u of
          Nothing => Just u
          Just u' => go f g u'
```

```
||| linear congruential generator, PCG multiplier
next64 : Bits64 -> Bits64
next64 s = s * 6364136223846793005 + 1442695040888963407

toI : Bits64 -> Int
toI x = cast x

rnd : Bits64 -> Int -> (Bits64, Int)
rnd s m =
  let s' = next64 s
      v  = (s' `div` 65536) `mod` (cast m)
  in (s', toI v)

rndN : Bits64 -> Nat -> (Bits64, Nat)
rndN s m = let (s', v) = rnd s (cast m) in (s', cast v)

||| uniform random binary tree with n leaves
gen : Bits64 -> Nat -> (Bits64, Tm)
gen s (S Z) = let (s', b) = rnd s 2
              in (s', if b == 0 then Sc else Kc)
gen s n =
  let (s1, l) = rndN s (minus n 1)
      (s2, tl) = gen s1 (S l)
      (s3, tr) = gen s2 (minus n (S l))
  in (s3, Ap tl tr)

genSized : Bits64 -> (Bits64, Tm)
genSized s = let (s1, k) = rndN s 6 in gen s1 (S (S k))

-- population helpers ------------------------------------------------

fill : IOArray Tm -> Int -> List Tm -> IO ()
fill arr i [] = pure ()
fill arr i (t :: ts) = do ignore (writeArray arr i t); fill arr (i + 1) ts

readAll : IOArray Tm -> Int -> Int -> IO (List Tm)
readAll arr i m =
  if i >= m then pure []
  else do Just t <- readArray arr i | Nothing => readAll arr (i+1) m
          ts <- readAll arr (i+1) m
          pure (t :: ts)

initA : Bits64 -> Int -> (Bits64, List Tm)
initA s 0 = (s, [])
initA s n = let (s1, t) = genSized s
                (s2, ts) = initA s1 (n - 1)
            in (s2, t :: ts)

seed22 : List Tm
seed22 =
  let atoms = [Sc, Kc]
      two = [Ap a b | a <- atoms, b <- atoms]
      thL = [Ap (Ap a b) c | a <- atoms, b <- atoms, c <- atoms]
      thR = [Ap a (Ap b c) | a <- atoms, b <- atoms, c <- atoms]
  in atoms ++ two ++ thL ++ thR
```

```
initB : Int -> List Tm
initB m = take (cast m) (concat (replicate 40 seed22))

-- the reactor ----------------------------------------------------

record Params where
  constructor MkP
  popM : Int
  fuel : Nat
  capIn : Nat
  capOut : Nat

||| one chunk of collisions; returns rng and elastic count
runChunk : Params -> IOArray Tm -> Bits64 -> Nat -> Nat -> IO (Bits64, Nat)
runChunk p arr s Z el = pure (s, el)
runChunk p arr s (S k) el = do
  let (s1, i) = rnd s (popM p)
  let (s2, j) = rnd s1 (popM p)
  Just ti <- readArray arr i | Nothing => runChunk p arr s2 k el
  Just tj <- readArray arr j | Nothing => runChunk p arr s2 k el
  if tsize ti + tsize tj > capIn p
    then runChunk p arr s2 k (S el)
    else case wnf (fuel p) (Ap ti tj) of
      Nothing => runChunk p arr s2 k (S el)
      Just pr =>
        if tsize pr > capOut p
          then runChunk p arr s2 k (S el)
          else do
            let (s3, r) = rnd s2 (popM p)
            ignore (writeArray arr r pr)
            runChunk p arr s3 k el

-- census and detectors -------------------------------------------

bump : String -> Nat -> SortedMap String (Nat, Nat) -> SortedMap String (Nat, Nat)
bump k sz m = case lookup k m of
  Nothing => insert k (1, sz) m
  Just (c, s) => insert k (c + 1, s) m

censusMap : List Tm -> SortedMap String (Nat, Nat)
censusMap = foldl (\m, t => bump (render t) (tsize t) m) empty

topK : Nat -> SortedMap String (Nat, Nat) -> List (String, Nat, Nat)
topK k m =
  take k (sortBy (\(_, c1, _), (_, c2, _) => compare c2 c1)
    (map (\(r, (c, s)) => (r, c, s)) (SortedMap.toList m)))

data Cls = CId | CConst | COther

classify : Nat -> Tm -> Cls
classify f t =
  case (wnf f (Ap t Sc), wnf f (Ap t Kc)) of
    (Just x, Just y) =>
      if render x == "S" && render y == "K" then CId
```

```
      else if render x == render y then CConst
      else COther
    _ => COther

countCls : Nat -> List Tm -> (Nat, Nat)
countCls f = foldl acc (0, 0)
  where acc : (Nat, Nat) -> Tm -> (Nat, Nat)
        acc (ic, cc) t = case classify f t of
          CId => (S ic, cc)
          CConst => (ic, S cc)
          COther => (ic, cc)

||| sample n random ordered pairs, count products landing on the
||| given target renders (production spectrum)
spectrum : Params -> IOArray Tm -> Bits64 -> Nat ->
           List String -> SortedMap String Nat -> IO (SortedMap String Nat)
spectrum p arr s Z tops m = pure m
spectrum p arr s (S k) tops m = do
  let (s1, i) = rnd s (popM p)
  let (s2, j) = rnd s1 (popM p)
  Just ti <- readArray arr i | Nothing => spectrum p arr s2 k tops m
  Just tj <- readArray arr j | Nothing => spectrum p arr s2 k tops m
  if tsize ti + tsize tj > capIn p
    then spectrum p arr s2 k tops m
    else case wnf (fuel p) (Ap ti tj) of
      Nothing => spectrum p arr s2 k tops m
      Just pr =>
        let r = render pr
        in if elem r tops
             then spectrum p arr s2 k tops
                    (case lookup r m of
                       Nothing => insert r 1 m
                       Just c => insert r (S c) m)
             else spectrum p arr s2 k tops m

shorten : String -> String
shorten str = if length str > 46 then substr 0 46 str ++ ".." else str

printCensus : String -> List Tm -> IO ()
printCensus tag pop = do
  let cm = censusMap pop
  let n = length (SortedMap.toList cm)
  let tot = foldl (\a, t => a + tsize t) 0 pop
  let (ic, cc) = countCls 250 pop
  putStrLn $ tag ++ " distinct=" ++ show n
           ++ " avgSize=" ++ show (div (the Int (cast tot)) (cast (length pop)))
           ++ " idLike=" ++ show ic ++ " constLike=" ++ show cc
  for_ (topK 8 cm) $ \(r, c, sz) =>
    putStrLn $ "  " ++ show c ++ "  sz" ++ show sz ++ "  " ++ shorten r

runExperiment : String -> Params -> Nat -> Nat -> Bits64 -> List Tm -> IO ()
runExperiment name p chunk nchunks s0 pop0 = do
  putStrLn $ "== " ++ name ++ " M=" ++ show (popM p)
           ++ " fuel=" ++ show (fuel p)
           ++ " capIn=" ++ show (capIn p) ++ " capOut=" ++ show (capOut p)
```

```
  arr <- newArray (popM p)
  fill arr 0 pop0
  pop <- readAll arr 0 (popM p)
  printCensus (name ++ " t=0") pop
  go arr s0 0 nchunks
  where
    go : IOArray Tm -> Bits64 -> Nat -> Nat -> IO ()
    go arr s done Z = do
      pop <- readAll arr 0 (popM p)
      let tops = map (\(r, _, _) => r) (topK 5 (censusMap pop))
      sp <- spectrum p arr (next64 (s + 99)) 20000 tops empty
      putStrLn $ name ++ " production spectrum over 20000 sampled pairs:"
      for_ tops $ \r =>
        putStrLn $ "  " ++ show (fromMaybe 0 (lookup r sp)) ++ "  " ++ shorten r
    go arr s done (S more) = do
      (s', el) <- runChunk p arr s chunk 0
      let done' = done + chunk
      pop <- readAll arr 0 (popM p)
      printCensus (name ++ " t=" ++ show done'
                   ++ " elastic=" ++ show el ++ "/" ++ show chunk) pop
      go arr s' done' more

main : IO ()
main = do
  let p = MkP 600 250 80 60
  let (sA, popA) = initA 42 600
  runExperiment "RUN-Afine(random-seed42)" p 2000 8 sA popA
  putStrLn ""
  let (sA2, popA2) = initA 42 600
  runExperiment "RUN-A(random-seed42)" p 30000 4 sA2 popA2
  putStrLn ""
  runExperiment "RUN-B(small-seed7)" p 30000 4 7 (initB 600)
  putStrLn ""
  let (sC, popC) = initA 1234 600
  runExperiment "RUN-C(random-seed1234)" p 30000 3 sC popC
```

Appendix C: The absorber section of the proof module

```
--------------------------------------------------------------------
-- Tier 2, part F: the soup absorber (E6)
--------------------------------------------------------------------

||| the term that fixated both soup runs of experiment E6
ssk : Term
ssk = App (App CS CS) CK

tau : Term
tau = App (App CS (App CK Proofs.ssk)) (App CK (App CK Proofs.ssk))

||| tau absorbs any argument in exactly four steps: one S firing
||| duplicates the partner, two K firings erase both copies, and a
||| final S firing rebuilds tau
tauAbsorbs : (x : Term) -> Steps (App Proofs.tau x) Proofs.tau
tauAbsorbs x =
  Then RedS (Then (CtxL RedK) (Then (CtxR RedK) (Then RedS Done)))
```

```
||| in the fixated monoculture every collision is tau with tau
tauFix : Steps (App Proofs.tau Proofs.tau) Proofs.tau
tauFix = tauAbsorbs Proofs.tau

||| tau takes no weak step: the absorber is a normal form
noStepS : Step CS u -> Void
noStepS p impossible

noStepK : Step CK u -> Void
noStepK p impossible

noStepSS : Step (App CS CS) u -> Void
noStepSS (CtxL p) = void (noStepS p)
noStepSS (CtxR p) = void (noStepS p)

noStepSSK : Step Proofs.ssk u -> Void
noStepSSK (CtxL p) = void (noStepSS p)
noStepSSK (CtxR p) = void (noStepK p)

noStepKSSK : Step (App CK Proofs.ssk) u -> Void
noStepKSSK (CtxL p) = void (noStepK p)
noStepKSSK (CtxR p) = void (noStepSSK p)

noStepKKSSK : Step (App CK (App CK Proofs.ssk)) u -> Void
noStepKKSSK (CtxL p) = void (noStepK p)
noStepKKSSK (CtxR p) = void (noStepKSSK p)

noStepTauL : Step (App CS (App CK Proofs.ssk)) u -> Void
noStepTauL (CtxL p) = void (noStepS p)
noStepTauL (CtxR p) = void (noStepKSSK p)

tauNormal : Step Proofs.tau u -> Void
tauNormal (CtxL p) = void (noStepTauL p)
tauNormal (CtxR p) = void (noStepKKSSK p)

||| the rival absorber that fixated the third run: eight steps,
||| four S firings and four K firings per absorption
upsA : Term
upsA = App (App CS (App CK CK)) Proofs.ssk

upsSA : Term
upsSA = App CS Proofs.upsA

ups : Term
ups = App (App CS Proofs.upsSA) (App CK Proofs.upsSA)

upsAbsorbs : (x : Term) -> Steps (App Proofs.ups x) Proofs.ups
upsAbsorbs x =
  Then RedS $ Then RedS $ Then (CtxL RedS) $ Then (CtxL (CtxL RedK)) $
  Then RedK $ Then RedS $ Then (CtxL (CtxR RedK)) $
  Then (CtxR (CtxR RedK)) Done

upsFix : Steps (App Proofs.ups Proofs.ups) Proofs.ups
upsFix = upsAbsorbs Proofs.ups
```

```
noStepKK : Step (App CK CK) u -> Void
noStepKK (CtxL p) = void (noStepK p)
noStepKK (CtxR p) = void (noStepK p)

noStepSKKh : Step (App CS (App CK CK)) u -> Void
noStepSKKh (CtxL p) = void (noStepS p)
noStepSKKh (CtxR p) = void (noStepKK p)

noStepUpsA : Step Proofs.upsA u -> Void
noStepUpsA (CtxL p) = void (noStepSKKh p)
noStepUpsA (CtxR p) = void (noStepSSK p)

noStepUpsSA : Step Proofs.upsSA u -> Void
noStepUpsSA (CtxL p) = void (noStepS p)
noStepUpsSA (CtxR p) = void (noStepUpsA p)

noStepKUpsSA : Step (App CK Proofs.upsSA) u -> Void
noStepKUpsSA (CtxL p) = void (noStepK p)
noStepKUpsSA (CtxR p) = void (noStepUpsSA p)

noStepUpsL : Step (App CS Proofs.upsSA) u -> Void
noStepUpsL (CtxL p) = void (noStepS p)
noStepUpsL (CtxR p) = void (noStepUpsSA p)

upsNormal : Step Proofs.ups u -> Void
upsNormal (CtxL p) = void (noStepUpsL p)
upsNormal (CtxR p) = void (noStepKUpsSA p)
```

Appendix D: The conserving reactor

```
||| Project secret3, experiment E6.2/C1 : the conserving reactor,
||| with an operational null (reactions off) and a logged reaction
||| network with food-closure and recycling-core analysis.
||| Conservation balances are compiled in Proofs.idr, part H.
module Soup2

import Data.IOArray
import Data.SortedMap
import Data.List
import Data.String
import Data.Maybe
import Data.IORef

%default covering

data Tm = Sc | Kc | Ap Tm Tm

render : Tm -> String
render Sc = "S"
render Kc = "K"
render (Ap a b) = render a ++ wrap b
  where wrap : Tm -> String
        wrap Sc = "S"
        wrap Kc = "K"
```

```
        wrap t = "(" ++ render t ++ ")"

tsize : Tm -> Nat
tsize Sc = 1
tsize Kc = 1
tsize (Ap a b) = tsize a + tsize b

next64 : Bits64 -> Bits64
next64 s = s * 6364136223846793005 + 1442695040888963407

rnd : Bits64 -> Int -> (Bits64, Int)
rnd s m =
  let s' = next64 s
      v  = (s' `div` 65536) `mod` (cast m)
  in (s', cast v)

data R1 = RK1 | RS1 Tm

redexes : Tm -> List (List Bool, R1)
redexes t0 = go [] t0
  where
    cls : Tm -> Maybe R1
    cls (Ap (Ap Kc _) _) = Just RK1
    cls (Ap (Ap (Ap Sc _) _) c) = Just (RS1 c)
    cls _ = Nothing
    go : List Bool -> Tm -> List (List Bool, R1)
    go p t@(Ap a b) =
      (case cls t of
         Just r => [(reverse p, r)]
         Nothing => [])
      ++ go (False :: p) a ++ go (True :: p) b
    go p _ = []

fireHead : Tm -> (Tm, List Tm)
fireHead (Ap (Ap Kc a) b) = (a, [b, Kc])
fireHead (Ap (Ap (Ap Sc a) b) c) = (Ap (Ap a c) (Ap b c), [Sc])
fireHead t = (t, [])

fireAt : List Bool -> Tm -> (Tm, List Tm)
fireAt [] t = fireHead t
fireAt (False :: p) (Ap a b) = let (a', fr) = fireAt p a in (Ap a' b, fr)
fireAt (True :: p) (Ap a b) = let (b', fr) = fireAt p b in (Ap a b', fr)
fireAt _ t = (t, [])

-- vessel ---------------------------------------------------------------

record Vessel where
  constructor MkV
  arr : IOArray Tm
  nRef : IORef Int

vN : Vessel -> IO Int
vN v = readIORef (nRef v)

vGet : Vessel -> Int -> IO Tm
```

```
vGet v i = do
  Just t <- readArray (arr v) i | Nothing => pure Kc
  pure t

vInsert : Vessel -> Tm -> IO ()
vInsert v t = do
  n <- readIORef (nRef v)
  ignore (writeArray (arr v) n t)
  writeIORef (nRef v) (n + 1)

vRemove : Vessel -> Int -> IO ()
vRemove v i = do
  n <- readIORef (nRef v)
  if i == n - 1
    then writeIORef (nRef v) (n - 1)
    else do last <- vGet v (n - 1)
            ignore (writeArray (arr v) i last)
            writeIORef (nRef v) (n - 1)

vRemove2 : Vessel -> Int -> Int -> IO ()
vRemove2 v i j =
  if i > j then do vRemove v i; vRemove v j
           else do vRemove v j; vRemove v i

readAll : Vessel -> IO (List Tm)
readAll v = do
  n <- vN v
  go 0 n
  where go : Int -> Int -> IO (List Tm)
        go i n = if i >= n then pure []
                 else do t <- vGet v i
                         ts <- go (i + 1) n
                         pure (t :: ts)

-- reaction network log ------------------------------------------------

Net : Type
Net = IORef (SortedMap String Nat)

bumpE : Net -> String -> String -> IO ()
bumpE net a b =
  modifyIORef net (\m =>
    let k = a ++ ">" ++ b
    in case lookup k m of
         Nothing => insert k 1 m
         Just c => insert k (S c) m)

-- reactions -----------------------------------------------------------

record Counters where
  constructor MkC
  cOk, cBlk, clv, clvBlk, kF, sF, sBlk, inert : IORef Nat

bumpR : IORef Nat -> IO ()
bumpR r = modifyIORef r S
```

```
probe : Vessel -> Bits64 -> Int -> Nat -> Nat -> String -> IO (Bits64, Maybe Int)
probe v s self Z zsz zr = pure (s, Nothing)
probe v s self (S k) zsz zr = do
  n <- vN v
  let (s1, j) = rnd s n
  if j == self
    then probe v s1 self k zsz zr
    else do u <- vGet v j
            if tsize u == zsz && render u == zr
              then pure (s1, Just j)
              else probe v s1 self k zsz zr

record Params where
  constructor MkP
  capMol : Nat
  probes : Nat
  reactOn : Bool

tick : Params -> Vessel -> Counters -> Net -> Bits64 -> IO Bits64
tick p v c net s = do
  n <- vN v
  let (s1, act) = rnd s 6
  if act <= 1
    then
      if n < 2 then do bumpR (cBlk c); pure s1
      else do
        let (s2, i) = rnd s1 n
        let (s3, j0) = rnd s2 (n - 1)
        let j = if j0 >= i then j0 + 1 else j0
        a <- vGet v i
        b <- vGet v j
        if tsize a + tsize b > capMol p
          then do bumpR (cBlk c); pure s3
          else do
            vRemove2 v i j
            let pr = Ap a b
            vInsert v pr
            bumpE net (render a) (render pr)
            bumpE net (render b) (render pr)
            bumpR (cOk c)
            pure s3
    else if act == 2
      then do
        let (s2, i) = rnd s1 n
        t <- vGet v i
        case t of
          Ap a b => do
            vRemove v i
            vInsert v a
            vInsert v b
            bumpE net (render t) (render a)
            bumpE net (render t) (render b)
            bumpR (clv c)
            pure s2
```

```
          _ => do bumpR (clvBlk c); pure s2
    else
      if not (reactOn p)
        then do bumpR (inert c); pure s1
        else do
          let (s2, i) = rnd s1 n
          t <- vGet v i
          case redexes t of
            [] => do bumpR (inert c); pure s2
            rs => do
              let (s3, ridx) = rnd s2 (cast (length rs))
              case drop (cast ridx) rs of
                [] => do bumpR (inert c); pure s3
                ((path, RK1) :: _) => do
                  let (t', freed) = fireAt path t
                  vRemove v i
                  vInsert v t'
                  traverse_ (vInsert v) freed
                  bumpE net (render t) (render t')
                  traverse_ (\u => bumpE net (render t) (render u)) freed
                  bumpR (kF c)
                  pure s3
                ((path, RS1 z) :: _) => do
                  (s4, hit) <- probe v s3 i (probes p) (tsize z) (render z)
                  case hit of
                    Nothing => do bumpR (sBlk c); pure s4
                    Just j => do
                      let (t', freed) = fireAt path t
                      vRemove2 v i j
                      vInsert v t'
                      traverse_ (vInsert v) freed
                      bumpE net (render t) (render t')
                      bumpE net (render z) (render t')
                      traverse_ (\u => bumpE net (render t) (render u)) freed
                      bumpR (sF c)
                      pure s4

-- census ----------------------------------------------------------------

bump : String -> Nat -> SortedMap String (Nat, Nat) -> SortedMap String (Nat, Nat)
bump k sz m = case lookup k m of
  Nothing => insert k (1, sz) m
  Just (cnt, s0) => insert k (cnt + 1, s0) m

censusMap : List Tm -> SortedMap String (Nat, Nat)
censusMap = foldl (\m, t => bump (render t) (tsize t) m) empty

censusList : SortedMap String (Nat, Nat) -> List (String, Nat, Nat)
censusList m =
  sortBy (\(_, c1, _), (_, c2, _) => compare c2 c1)
    (map (\(r, (cn, sz)) => (r, cn, sz)) (SortedMap.toList m))

shorten : String -> String
shorten str = if length str > 44 then substr 0 44 str ++ ".." else str
```

```
cGet : SortedMap String (Nat, Nat) -> String -> Nat
cGet m r = fst (fromMaybe (0, 0) (lookup r m))

printCensus : String -> Vessel -> Counters -> IO ()
printCensus tag v c = do
  pop <- readAll v
  let cm = censusMap pop
  let mass = foldl (\a, t => a + tsize t) 0 pop
  co <- readIORef (cOk c); cb <- readIORef (cBlk c)
  cl <- readIORef (clv c); cbl <- readIORef (clvBlk c)
  kf <- readIORef (kF c); sf <- readIORef (sF c)
  sb <- readIORef (sBlk c); iw <- readIORef (inert c)
  putStrLn $ tag ++ " N=" ++ show (length pop)
    ++ " distinct=" ++ show (length (SortedMap.toList cm))
    ++ " mass=" ++ show mass ++ (if mass == 6000 then "" else " MASS-ERROR")
    ++ " freeS=" ++ show (cGet cm "S") ++ " freeK=" ++ show (cGet cm "K")
  putStrLn $ "  cond=" ++ show co ++ " condBlk=" ++ show cb
    ++ " cleave=" ++ show cl ++ " cleaveBlk=" ++ show cbl
    ++ " kFire=" ++ show kf ++ " sFire=" ++ show sf
    ++ " sBlocked=" ++ show sb ++ " inert=" ++ show iw
  for_ (take 8 (censusList cm)) $ \(r, cn, sz) =>
    putStrLn $ "  " ++ show cn ++ "  sz" ++ show sz ++ "  " ++ shorten r

printFinal : String -> Vessel -> IO ()
printFinal tag v = do
  pop <- readAll v
  let cm = censusMap pop
  let rows = filter (\(_, cn, _) => cn >= 2) (censusList cm)
  putStrLn $ tag ++ " final species table (count >= 2), "
    ++ show (length rows) ++ " species:"
  for_ (take 24 rows) $ \(r, cn, sz) =>
    putStrLn $ "  " ++ show cn ++ "  sz" ++ show sz ++ "  " ++ shorten r

-- network analysis ----------------------------------------------------

splitEdge : String -> (String, String)
splitEdge s =
  let (a, b) = break (== '>') s
  in (a, pack (drop 1 (unpack b)))

addAdj : String -> String -> SortedMap String (List String) ->
         SortedMap String (List String)
addAdj a b m = case lookup a m of
  Nothing => insert a [b] m
  Just xs => if elem b xs then m else insert a (b :: xs) m

bfs : SortedMap String (List String) -> List String ->
      SortedMap String Bool -> SortedMap String Bool
bfs adj [] vis = vis
bfs adj (x :: xs) vis =
  case lookup x vis of
    Just _ => bfs adj xs vis
    Nothing =>
      let nbrs = fromMaybe [] (lookup x adj)
      in bfs adj (nbrs ++ xs) (insert x True vis)
```

```
analyzeNet : String -> Vessel -> Net -> IO ()
analyzeNet tag v net = do
  m <- readIORef net
  let es = map (\(k, c) => let (a, b) = splitEdge k in (a, b, c))
               (SortedMap.toList m)
  let fwd = foldl (\acc, (a, b, _) => addAdj a b acc)
                  (the (SortedMap String (List String)) empty) es
  let rev = foldl (\acc, (a, b, _) => addAdj b a acc)
                  (the (SortedMap String (List String)) empty) es
  let fc = bfs fwd ["S", "K"] (the (SortedMap String Bool) empty)
  let rc = bfs rev ["S", "K"] (the (SortedMap String Bool) empty)
  pop <- readAll v
  let cm = censusMap pop
  let species = censusList cm
  let unreach = filter (\(r, _, _) => isNothing (lookup r fc)) species
  let core = filter (\(r, _, _) =>
                isJust (lookup r fc) && isJust (lookup r rc)) species
  putStrLn $ tag ++ " network: edges=" ++ show (length es)
    ++ " speciesNow=" ++ show (length species)
    ++ " foodReachable=" ++ show (length species `minus` length unreach)
    ++ " unreachable=" ++ show (length unreach)
    ++ " recyclingCore=" ++ show (length core)
  putStrLn $ tag ++ " recycling core, top by abundance:"
  for_ (take 14 core) $ \(r, cn, sz) =>
    putStrLn $ "  " ++ show cn ++ "  sz" ++ show sz ++ "  " ++ shorten r
  let tops = take 10 (sortBy (\(_, _, c1), (_, _, c2) => compare c2 c1) es)
  putStrLn $ tag ++ " top edges by flux:"
  for_ tops $ \(a, b, c) =>
    putStrLn $ "  " ++ show c ++ "  " ++ shorten a ++ " > " ++ shorten b

runE : String -> Params -> Bits64 -> IO ()
runE name p s0 = do
  arr0 <- newArray 6100
  nR <- newIORef 0
  let v = MkV arr0 nR
  for_ (the (List Int) [0..2999]) $ \_ => vInsert v Sc
  for_ (the (List Int) [0..2999]) $ \_ => vInsert v Kc
  r1 <- newIORef 0; r2 <- newIORef 0; r3 <- newIORef 0; r4 <- newIORef 0
  r5 <- newIORef 0; r6 <- newIORef 0; r7 <- newIORef 0; r8 <- newIORef 0
  let c = MkC r1 r2 r3 r4 r5 r6 r7 r8
  net <- newIORef empty
  putStrLn $ "== " ++ name ++ " capMol=" ++ show (capMol p)
    ++ " probes=" ++ show (probes p)
    ++ " reactions=" ++ (if reactOn p then "on" else "off")
  go v c net s0 0 6
  printFinal name v
  analyzeNet name v net
  where
    loop : Vessel -> Counters -> Net -> Bits64 -> Nat -> IO Bits64
    loop v c net s Z = pure s
    loop v c net s (S k) = do s' <- tick p v c net s; loop v c net s' k
    go : Vessel -> Counters -> Net -> Bits64 -> Nat -> Nat -> IO ()
    go v c net s done Z = pure ()
    go v c net s done (S more) = do
```

```
      s' <- loop v c net s 30000
      let done' = done + 30000
      printCensus (name ++ " t=" ++ show done') v c
      go v c net s' done' more

main : IO ()
main = do
  runE "FULL-D(seed11)" (MkP 24 24 True) 11
  putStrLn ""
  runE "FULL-E(seed77)" (MkP 24 24 True) 77
  putStrLn ""
  runE "NULL-D(seed11)" (MkP 24 24 False) 11
  putStrLn ""
  runE "NULL-E(seed77)" (MkP 24 24 False) 77
```

Appendix E: The conservation section of the proof module

```
----------------------------------------------------------------------
-- Tier 2, part H: the conserving chemistry balances (E6.2)
----------------------------------------------------------------------

||| atoms in a term: every leaf counts one
tSize : Term -> Nat
tSize (Var _)   = 1
tSize CK        = 1
tSize CS        = 1
tSize (App a b) = tSize a + tSize b

sCount : Term -> Nat
sCount (Var _)   = 0
sCount CK        = 0
sCount CS        = 1
sCount (App a b) = sCount a + sCount b

kCount : Term -> Nat
kCount (Var _)   = 0
kCount CK        = 1
kCount CS        = 0
kCount (App a b) = kCount a + kCount b

||| condensation conserves mass on the nose
condMass : (a, b : Term) -> tSize (App a b) = tSize a + tSize b
condMass a b = Refl

||| the K reaction balances: the redex weighs exactly its product
||| plus the freed argument plus one released K atom
kBalanced : (a, b : Term) ->
  tSize (App (App CK a) b) = tSize a + tSize b + 1
kBalanced a b = plusCommutative 1 (tSize a + tSize b)

kBalK : (a, b : Term) ->
  kCount (App (App CK a) b) = kCount a + kCount b + 1
kBalK a b = plusCommutative 1 (kCount a + kCount b)

kBalS : (a, b : Term) ->
```

```
  sCount (App (App CK a) b) = sCount a + sCount b
kBalS a b = Refl

||| the S reaction balances: the redex plus one recruited copy of the
||| shared argument weighs exactly its product plus one released S atom
sBalanced : (a, b, c : Term) ->
  tSize (App (App (App CS a) b) c) + tSize c
    = tSize (App (App a c) (App b c)) + 1
sBalanced a b c =
  trans (cong S (trans (sym (plusAssociative (tSize a + tSize b) (tSize c) (tSize
c)))
                       (plusSwapMid (tSize a) (tSize b) (tSize c) (tSize c))))
        (plusCommutative 1 ((tSize a + tSize c) + (tSize b + tSize c)))

sBalS : (a, b, c : Term) ->
  sCount (App (App (App CS a) b) c) + sCount c
    = sCount (App (App a c) (App b c)) + 1
sBalS a b c =
  trans (cong S (trans (sym (plusAssociative (sCount a + sCount b) (sCount c)
(sCount c)))
                       (plusSwapMid (sCount a) (sCount b) (sCount c) (sCount c))))
        (plusCommutative 1 ((sCount a + sCount c) + (sCount b + sCount c)))

sBalK : (a, b, c : Term) ->
  kCount (App (App (App CS a) b) c) + kCount c
    = kCount (App (App a c) (App b c))
sBalK a b c =
  trans (sym (plusAssociative (kCount a + kCount b) (kCount c) (kCount c)))
        (plusSwapMid (kCount a) (kCount b) (kCount c) (kCount c))
```

References

1. Project secret1, 'A Tree-Based Number System over the Combinators K and S, and Primality as Unreachability', working paper v0.4, August 11, 2026.
2. Project secret2, 'Transparent Arithmetic over the Combinators K and S: Divisibility Automata, Factoring as Rewriting, and the Search for a Prime-Revealing Representation', working paper v0.8, August 11, 2026.
3. Project secret3, 'The Linearity Ladder: Where Erasure and Duplication Enter a Two-Combinator Arithmetic', working paper v0.1, August 11, 2026.
4. Project secret3, 'The Reach of K and S: A Program Note on Demonstrations in Mathematics and Physics', program note v0.4, August 11, 2026.
5. W. Fontana and L. W. Buss, "The arrival of the fittest': toward a theory of biological organization', Bulletin of Mathematical Biology 56 (1994), 1-64.
6. G. Kruszewski and T. Mikolov, 'Combinatory chemistry: towards a simple model of emergent evolution', Proceedings of ALIFE 2020, MIT Press, 2020.
7. E. Brady, 'Idris 2: Quantitative Type Theory in Practice', *Proceedings of ECOOP 2021*, LIPIcs 194, 2021.